EduSpace · Guion por diapositiva

Qué dice cada integrante, qué muestra y cómo pasa la palabra · Defensa DIST 2026-2 · ~45 minutos

Quién	Diapositivas	Tiempo propio
Hermes	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 24	15 min 30 s
Andre	13, 14, 15, 16, 17	9 min 00 s
Witmer	18, 19, 20, 21, 22, 25	10 min 00 s
Todos	23, 26	(cada quien dice su parte)

Cómo usar este guion

- La numeración sigue **docs/presentacion/04-diapositivas-para-generador.md**: la diapositiva N de este PDF es la N de la presentación.
- **Qué decir** está escrito para decirse en voz alta, con tus palabras: no se lee de corrido, se usa como guía.
- **Qué mostrar** indica qué señalar, abrir o ejecutar. **Pase** es la frase que le da la palabra al siguiente.
- Los tiempos son de referencia (suman ~40 min más ~5 de preguntas). Si falta tiempo, recorta la demostración, nunca la explicación del proceso.

Reparto propuesto. Hermes modera la apertura (1–7) y la seguridad (24); Witmer lleva las métricas (25); las diapositivas 23 y 26 las dicen los tres. Si prefieren otro reparto, solo cambien el nombre en cada tarjeta: el texto sirve para cualquiera.

Antes de empezar (15 minutos antes)

- Sistema levantado y 10 servicios en **UP** (Eureka). Web en **localhost:4200**; Mailpit en **localhost:18025**.
- `python3 tools/demo/limpiar_pruebas.py` y luego `python3 tools/demo/preparar_exposicion.py`.
- `python3 tools/demo/preparar_matricula_en_linea.py`: da los datos nuevos para la matrícula en línea (alumno, apoderado, correo) y libera cupos.
- Caja del cajero **abierta** (la usa la demostración de Witmer).
- Cada quien ensayó su script una vez hoy: `proceso_hermes.py`, `proceso_andre.py`, `proceso_witmer.py`.
- Capturas de respaldo guardadas por si falla la red.

Apertura · diapositivas 1–7 (6 min)

1 · Portada · EduSpace

Hermes 0:30 min

QUÉ DECIR

Buenos días. Somos Hermes, Andre Fabricio y Witmer, y les presentamos EduSpace: un sistema de gestión escolar distribuido para el colegio Horizonte del Valle. Son diez microservicios que trabajan juntos. Cada uno de nosotros defenderá tres servicios y un proceso transaccional de su parte, con demostración en vivo.

QUÉ MOSTRAR

Portada. Nada más; mirar a la clase.

PASE A LA SIGUIENTE

Empiezo por el problema que queríamos resolver.

2 · El problema · El colegio vive en hojas sueltas

Hermes 1:00 min

QUÉ DECIR

En un colegio, la matrícula, la asistencia, las notas, los cobros y los horarios suelen vivir en listas separadas. Un mismo alumno existe en cinco listas distintas, y cuando una cambia, las otras no se enteran. Las familias se enteran tarde y cada área tiene su propia verdad. Ese es el problema de fondo: datos duplicados y desfasados.

QUÉ MOSTRAR

Diagrama de cinco cajas desconectadas con flechas rotas.

PASE A LA SIGUIENTE

Veamos cómo lo resolvemos.

3 · La solución · Una verdad por dato, eventos para el resto

Hermes 1:00 min

QUÉ DECIR

Nuestra idea: cada dato tiene un solo dueño. Son diez microservicios con base de datos propia. Se comunican de dos maneras: consultas internas síncronas y eventos por Kafka. Todo entra por un Gateway único, y hay un portal por rol y un sitio público. Ningún servicio lee la base de otro: lo que necesita le llega por evento o por consulta interna.

QUÉ MOSTRAR

Las mismas cinco cajas, ahora unidas por una línea central (Kafka) y el Gateway arriba.

PASE A LA SIGUIENTE

Ahora los números, para dimensionar el sistema.

4 · EduSpace en cifras

Hermes 0:45 min

QUÉ DECIR

Son diez servicios más Config Server, Eureka y el Gateway. Superamos los 300 endpoints, tenemos 25 tópicos de Kafka, cerca de 280 pruebas de backend y 65 de frontend, y auditamos más de cien pantallas. Las cifras están medidas y documentadas; el detalle está en la documentación de métricas.

QUÉ MOSTRAR

Cinco tarjetas con número grande. No leer todas: resaltar servicios, tópicos y pruebas.

PASE A LA SIGUIENTE

Veamos cómo encajan las piezas.

5 · Arquitectura distribuida

Hermes 1:15 min

QUÉ DECIR

El navegador, o la app móvil, solo habla con el Gateway, en el puerto 18080. El Gateway valida el token JWT, aplica límites por IP y enruta a los servicios. Eureka descubre dónde está cada servicio y el Config Server centraliza la configuración. Cada servicio tiene su PostgreSQL, los archivos van a MinIO y los cambios se comunican por Kafka. Y cada flecha tiene un contrato: HTTP interno con un token de servicio, o un evento con sobre estándar.

QUÉ MOSTRAR

Señalar con el puntero: Navegador → Gateway → servicios; luego Kafka al centro y las bases abajo.

PASE A LA SIGUIENTE

Ahora, cómo repartimos el trabajo.

6 · Reparto del equipo · Quién defiende qué

Hermes 0:30 min

QUÉ DECIR

Hermes defiende planificación, alumnos y notificaciones, además de las cuentas de acceso. Andre Fabricio, matrícula, asistencia y tareas. Witmer, pago, exámenes y calificaciones. La infraestructura, la seguridad y la observabilidad son de los tres. Cada uno mostrará un proceso transaccional de su parte.

QUÉ MOSTRAR

Tabla de tres columnas con la cinta de color de cada uno.

PASE A LA SIGUIENTE

Antes de entrar en cada parte, una idea que nos une a los tres.

7 · Las tres ideas de consistencia

Hermes 1:00 min

QUÉ DECIR

Como cada servicio tiene su base, no hay una transacción global. Mantenemos la consistencia con tres ideas. Una: el outbox, el dato y su evento se guardan en la misma transacción. Dos: la idempotencia, repetir una operación no la duplica. Tres: la compensación, si algo sale mal se deshace con otro paso, no con un rollback global. Es la consistencia distribuida con transacciones locales, eventos y compensaciones. Cada proceso que van a ver usa estas tres ideas.

QUÉ MOSTRAR

Tres iconos grandes: caja de salida, doble check y flecha de deshacer.

PASE A LA SIGUIENTE

Empiezo yo: planificación, alumnos y notificaciones.

Bloque de Hermes · 8–12 (9 min)

8 · Hermes · planificación, alumnos, notificaciones

Hermes 1:30 min

QUÉ DECIR

Mi parte. Planificación guarda el año escolar, las secciones, la malla curricular, los docentes y los horarios, y es la fuente de verdad que casi todos consultan. Alumnos guarda el expediente, los apoderados y los documentos. Notificaciones se ocupa de la bandeja, el correo y las plantillas. Y el servicio de autenticación crea las cuentas con el formato nombre.apellido. Aquí se ven el Panel de aulas y el Armado automático de horarios.

QUÉ MOSTRAR

Capturas del Panel de aulas y del Armado automático, lado a lado.

PASE A LA SIGUIENTE

Mi proceso transaccional: publicar un horario «todo o nada».

9 · Publicar un horario «todo o nada»

Hermes 2:00 min

QUÉ DECIR

Generar un horario toca cientos de filas, y no puede quedar a medias. El flujo es: primero simular, que es la misma transacción pero revertida, así que nada se guarda. Luego generar sin cruces, validar y publicar. Si hay un choque, por ejemplo un docente con dos clases a la misma hora, el sistema responde 409 y no deja nada a medias. Al publicar, la nueva versión del horario y el evento horario.publicado se guardan en una sola transacción, y Kafka lo entrega a notificaciones, que avisa a docentes y familias.

QUÉ MOSTRAR

Flujo: Simular (etiqueta roja «nada se guarda») → Generar → Validar → Publicar → versión + evento → Kafka → avisos.

PASE A LA SIGUIENTE

Les muestro el código que lo hace posible.

10 · El código clave

Hermes 1:30 min

QUÉ DECIR

Dos fragmentos. En el generador de horarios, cuando se pide simular, marcamos la transacción como solo-rollback: se hace todo el cálculo, pero al final se deshace. Y en el registrador del outbox, un listener que corre justo antes del commit inserta el evento en la tabla outbox_eventos, dentro de la misma transacción del dato. Si el commit falla, el evento tampoco existe; si el commit funciona, el evento está garantizado.

QUÉ MOSTRAR

Abrir en el editor la línea 144 del generador y luego el OutboxRecorder. Resaltar setRollbackOnly y BEFORE_COMMIT.

PASE A LA SIGUIENTE

Vamos a verlo funcionar.

11 · Demostración en vivo

Hermes 2:00 min

QUÉ DECIR

Ejecuto el script del proceso. Primero simula: coloca 288 clases en 12 secciones y, al terminar, la base sigue en 288, sin cambios. Después intenta duplicar una clase del mismo docente en el mismo bloque y recibe el 409: las clases de la sección siguen en 24, nada a medias. Por último publica: se crea la versión 2 y el evento queda en el outbox. Ahora abro el portal y muestro el aviso que recibe el docente.

QUÉ MOSTRAR

Terminal: python3 tools/demo/proceso_hermes.py. Resaltar «SIN CAMBIOS (rollback)», «HTTP 409» y «versión publicada». Luego mostrar el aviso en Notificaciones.

PASE A LA SIGUIENTE

Resumo lo que aprendimos.

12 · Qué aprendimos (Hermes)

Hermes 1:00 min

QUÉ DECIR

Tres lecciones. El rollback nos permite simular con seguridad: lo que veo es lo que se guardaría. El outbox evita perder avisos aunque Kafka esté caído, porque el evento espera en la base. Y los consumidores deben ser idempotentes, porque un mismo evento puede llegar más de una vez. Medimos 12 secciones, 288 clases, cero cruces, en menos de un segundo. Le paso la palabra a Andre.

QUÉ MOSTRAR

Frase destacada: «Lo que veo es lo que se guardaría.»

PASE A LA SIGUIENTE

Andre, tu turno.

Bloque de Andre · 13–17 (9 min)

13 · Andre · matrícula, asistencia, tareas

Andre 1:30 min

QUÉ DECIR

Gracias, Hermes. Yo defiendo matrícula, asistencia y tareas. Matrícula maneja las campañas, los cupos, los estados, el traslado y el retiro, y es la dueña de la relación alumno–sección. Asistencia lleva sesiones, justificaciones y reportes. Tareas, las entregas con archivos y la calificación de 0 a 20. Como no pueden consultar a matrícula en cada petición, asistencia y tareas guardan una réplica local, matriculas_local, que se alimenta por eventos.

QUÉ MOSTRAR

Capturas de Cupos y del curso del docente (pestaña Tareas).

PASE A LA SIGUIENTE

Mi proceso: la matrícula con cupo atómico, pago, aprobación y compensación.

14 · Matrícula: cupo atómico, pago y compensación

Andre 2:30 min

QUÉ DECIR

Es una saga coreografiada: cada paso es una transacción local y los servicios reaccionan a eventos. Primero, una transacción reserva el cupo y crea la pre-matrícula, y publica matricula.creada. Pago crea la cuota y cobra. Cuando llega pago.confirmado, la matrícula pasa a POR_APROBAR: ya pagó, el cupo está reservado, pero todavía no es una matrícula. Entonces el administrador la revisa y la aprueba; recién ahí pasa a MATRICULADO y se publica matricula.confirmada, que genera las diez pensiones y habilita al alumno en asistencia y tareas. Y las compensaciones, en rojo: si la reserva vence sin pago, o se anula, o se retira, se publica el evento y el cupo vuelve y las cuotas pendientes se anulan.

QUÉ MOSTRAR

Diagrama de secuencia en tres carriles (matrícula · Kafka · pago). Señalar el recuadro de la aprobación del administrador y las flechas rojas punteadas.

PASE A LA SIGUIENTE

Veamos el código que protege el cupo.

15 · El código clave

Andre 1:30 min

QUÉ DECIR

Dos ideas. La primera: reservar el cupo es una sola sentencia, UPDATE cupos_seccion SET ocupados = ocupados + 1 WHERE ocupados < capacidad. Comprueba y reserva a la vez; si ya no hay lugar, devuelve cero filas y respondemos 409. No hay carrera por la última vacante. La segunda: el servicio del outbox está marcado con Propagation.MANDATORY, es decir, obliga a que el evento viaje dentro de la transacción del dato; no se puede publicar por fuera.

QUÉ MOSTRAR

Abrir CupoSeccionRepository (resaltar el WHERE) y EventoOutboxService (resaltar MANDATORY).

PASE A LA SIGUIENTE

Lo demuestro con el script.

16 · Demostración en vivo

Andre 2:30 min

QUÉ DECIR

Ejecuto el proceso. Inicia una matrícula en línea y reserva el cupo; se ve cuántos cupos ocupados hay. Pago con la tarjeta de rechazo: HTTP 402, no se cobra y la reserva sigue intacta. Pago con la tarjeta buena: la matrícula queda POR_APROBAR, sin cuentas ni pensiones todavía; repetir el pago devuelve el mismo comprobante, no cobra dos veces. Secretaría intenta aprobar y recibe 403; el administrador aprueba, y ahora sí: cuentas, diez pensiones y el correo con el acceso, que vemos en Mailpit. Por último, retiro: el cupo vuelve.

QUÉ MOSTRAR

Terminal: AUTH_MODE=eduspace python3 tools/demo/proceso_andre.py. Resaltar 402, POR_APROBAR, 403 de secretaría, «pensiones=10» y los cupos antes/después. Luego Kafka UI y Mailpit.

PASE A LA SIGUIENTE

Cierro con lo aprendido.

17 · Qué aprendimos (Andre)

Andre 1:00 min

QUÉ DECIR

La concurrencia se resuelve en la base de datos, no en el código: el cupo es una sentencia, no una esperanza. La idempotencia hace que el pago repetido no duplique nada. Y lo que no se paga, se libera: la reserva vence y devuelve el cupo. Lo probamos con dos hilos peleando por el último cupo, pagos repetidos, vencimiento y Kafka caído. Witmer continúa con el dinero.

QUÉ MOSTRAR

Frase destacada: «El cupo es una sentencia, no una esperanza.»

PASE A LA SIGUIENTE

Witmer, tu turno.

Bloque de Witmer · 18–22 (9 min)

18 · Witmer · pago, exámenes, calificaciones

Witmer 1:30 min

QUÉ DECIR

Gracias, Andre. Yo defiendo pago, exámenes y calificaciones. Pago es el dueño del dinero: conceptos, tarifas, cuotas, caja, comprobantes y mora; aquí un error cuesta caro. Exámenes controla el intento con el tiempo del servidor, y nunca envía al navegador cuál es la respuesta correcta. Calificaciones arma los criterios y promedios, el cierre de periodo y el boletín en PDF; las familias solo ven los periodos cerrados.

QUÉ MOSTRAR

Capturas de Finanzas → Resumen y de la libreta/boletín.

PASE A LA SIGUIENTE

Mi proceso: el cobro idempotente.

19 · Cobro idempotente con comprobante y evento

Witmer 2:30 min

QUÉ DECIR

Un cobro llega con una clave de idempotencia. Primero pregunto: ¿ya existe un pago con esta clave? Si sí, devuelvo el mismo pago, no cobro de nuevo; así un reintento de red no duplica. Si no existe, todo ocurre en una sola transacción: valido, registro el pago y su detalle, actualizo los saldos con control de versión, tomo el siguiente número de comprobante con un bloqueo, y guardo el evento en el outbox. Y la compensación: anular un pago no borra nada; revierte los saldos, anula el comprobante y publica pago.anulado para que matrícula y notificaciones se enteren.

QUÉ MOSTRAR

Flujo: POST /pagos + idempotencyKey → «¿ya existe?» → recuadro «UNA transacción» → Kafka. Abajo, flecha roja de Anular.

PASE A LA SIGUIENTE

Les muestro tres fragmentos de código.

20 · El código clave

Witmer 1:30 min

QUÉ DECIR

Tres ideas en tres líneas. Una: findByIdempotencyKey, si ya existe devuelvo ese pago. Dos: findFirstByTipoActivaForUpdate, bloquea la fila de la serie del comprobante mientras tomo el número, así dos cobros simultáneos no repiten el mismo, y no hay huecos. Tres: en la anulación, cuota.setSaldo(saldo + montoAplicado): la compensación restituye el saldo.

QUÉ MOSTRAR

Abrir PagoService, ComprobanteService y AnulacionService; resaltar las tres líneas.

PASE A LA SIGUIENTE

Vamos a ejecutarlo.

21 · Demostración en vivo

Witmer 2:00 min

QUÉ DECIR

Ejecuto el script con la caja abierta. Primero un cobro: sale el comprobante y baja el saldo. Repito con la misma clave: mismo pago, mismo comprobante, el saldo no cambia. Intento cobrar más que el saldo: 409 y el saldo queda intacto. Y anulo el pago: el saldo se restituye y se publica pago.anulado.

QUÉ MOSTRAR

Terminal: AUTH_MODE=eduspace python3 tools/demo/proceso_witmer.py. Resaltar «mismo pago: True», el 409 y «saldo restituido». Después cerrar la caja desde la pantalla Caja.

PASE A LA SIGUIENTE

Cierro con las lecciones.

22 · Qué aprendimos (Witmer)

Witmer 1:00 min

QUÉ DECIR

La clave de idempotencia protege del reintento; el bloqueo del correlativo evita comprobantes duplicados; y anular no borra, compensa. Cobrar dos veces no es un error de red: es un error de diseño. Lo probamos con dos cobros simultáneos, comprobante único y anulación. Ahora vamos a ver todo junto.

QUÉ MOSTRAR

Frase destacada: «Cobrar dos veces no es un error de red: es un error de diseño.»

PASE A LA SIGUIENTE

Los tres: el proceso integrado.

Cierre conjunto · 23–26 (5 min + preguntas)

23 · Matrícula en línea, de punta a punta

Todos 3:00 min

QUÉ DECIR

Andre: «Una familia entra al sitio público, sin sesión, elige el grado y llena los datos; se reserva el cupo.» Witmer: «Paga con la pasarela simulada; el cobro es idempotente y genera su comprobante.» Hermes: «El pago deja la matrícula en revisión. El administrador la aprueba y entonces se crean las cuentas con nombre.apellido, se generan las pensiones y se publica el horario.» Andre: «Y la familia recibe un correo con su usuario, la clave inicial, la constancia en PDF, el horario y las finanzas. Al entrar por primera vez, el sistema le pide cambiar la clave.»

QUÉ MOSTRAR

En el navegador: sitio público → pagar → pantalla «en revisión» → (otra pestaña, admin) Matrícula → Por aprobar → Aprobar → Mailpit con el correo → ingresar con la cuenta nueva.

PASE A LA SIGUIENTE

Veamos cómo lo protegemos.

24 · Seguridad y resiliencia

Hermes 1:30 min

QUÉ DECIR

Los tokens JWT los emite Keycloak o nuestro servicio de autenticación. Los endpoints internos solo aceptan un token de servicio. El Gateway limita las peticiones por IP, con un cupo especial para el sitio público. En desarrollo la clave de firma es persistente, y si un servicio se reinicia o la sesión deja de valer, la web muestra la franja «servicio caído» y lleva de vuelta al inicio de sesión, en lugar de quedarse con datos viejos.

QUÉ MOSTRAR

Candado y escudo; captura de la franja de aviso. Si da tiempo: reiniciar auth-ms con la sesión abierta y mostrar que el token sigue valiendo.

PASE A LA SIGUIENTE

Y los números que lo respaldan.

25 · Lo medimos

Witmer 1:30 min

QUÉ DECIR

La latencia mediana de las consultas está entre 10 y 70 milisegundos. El horario de 288 clases se genera en 0,3 a 0,6 segundos. El lag de Kafka es cero. Tenemos cerca de 280 pruebas de backend y 65 de frontend, y auditamos más de cien pantallas. Son mediciones en entorno de desarrollo; las cifras y su método están en la documentación de métricas.

QUÉ MOSTRAR

Tabla pequeña de latencias con indicadores verdes.

PASE A LA SIGUIENTE

Cerramos con lo que falta.

26 · Lo que falta y lo que sigue

Todos 2:00 min

QUÉ DECIR

Hermes: «Falta una pasarela de pago real y el correo real por SMTP.» Andre: «Falta el despliegue en producción con pruebas de carga y definir el prorrateo de pensiones cuando se matricula a mitad de año.» Witmer: «Y reembolsos automáticos al rechazar una matrícula.» Todos: «Para cerrar: una familia matricula desde la web y todo ocurre por transacciones locales y eventos; si algo falla, el sistema compensa. Gracias, ¿preguntas?»

QUÉ MOSTRAR

Línea «Hoy / Siguiente» y el contacto del equipo.

PASE A LA SIGUIENTE

Preguntas (ver la hoja de respuestas).

Versión corta · demostración de 10 minutos

Si el tiempo es corto, se presentan estas 5 escenas con la plataforma en vivo (guion completo en **docs/presentacion/05-demo-rapida.md**). Credenciales y comandos de preparación están ahí; todas las cuentas de la demo usan la clave **Expo12345**.

Escena 1 · Proceso 1 · Nuevo alumno y aprobación

Andre 3:00 min

QUÉ DECIR

Una familia nueva quiere matricular a su hija. Entra al sitio, elige grado, llena los datos y paga con la tarjeta de prueba. Se reserva la vacante y la página dice «Pago recibido, tu matrícula está en revisión»: todavía no hay usuario ni clave. Llega el primer correo, de pago recibido. Ahora el administrador, en Matrícula → Por aprobar, revisa el origen, el pago y el expediente, y aprueba. Solo el administrador puede: secretaría recibe 403. Llega el segundo correo: matrícula aprobada, con un recuadro «Para iniciar sesión», la constancia en PDF, el horario y las pensiones. Entro con el usuario nuevo y el sistema me pide cambiar la clave.

QUÉ MOSTRAR

Sitio público → Mailpit → Matrícula → Por aprobar → Mailpit → login con la cuenta nueva.

PASE A LA SIGUIENTE

Pasemos al aula.

Escena 2 · Proceso 2 · Profesor en clase

Hermes 2:00 min

QUÉ DECIR

El docente abre su app y ve la clase de hoy, que viene del horario que publicó planificación. La toca y pasa lista: presente para todos y una falta. Guarda y cierra; queda bloqueada. Luego sube un material del curso, un PDF, y crea y publica una tarea. La familia recibe el aviso en el momento.

QUÉ MOSTRAR

App móvil del docente: Hoy → Pasar lista → Cursos → Materiales → Nueva tarea.

PASE A LA SIGUIENTE

Ahora la alumna.

Escena 3 · Proceso 3 · Alumna: tarea y examen

Witmer 2:00 min

QUÉ DECIR

La alumna ve la tarea, adjunta un archivo y la entrega; el archivo sube directo al almacenamiento con una URL firmada. El docente la califica. Luego la alumna rinde el examen: el servidor controla el tiempo y nunca le manda cuál es la respuesta correcta. Al enviar, se califica automáticamente.

QUÉ MOSTRAR

Web de la alumna: curso → Tareas → Entregar; docente → Calificar; alumna → Exámenes → Rendir.

PASE A LA SIGUIENTE

Y la familia, ¿qué ve?

Escena 4 · Proceso 4 · Apoderada

Hermes 1:00 min

QUÉ DECIR

La apoderada ve a su hija: el resumen, la asistencia con la falta que acabamos de marcar, las notas del bimestre cerrado, el boletín en PDF y las cuotas, que puede pagar. Todo viene de cuatro servicios distintos, y cada familia solo ve lo suyo.

QUÉ MOSTRAR

Móvil o web de la apoderada: Inicio → Asistencia → Notas → Pagos.

PASE A LA SIGUIENTE

Por último, el control del colegio.

Escena 5 · Proceso 5 · Administración y finanzas

Witmer 2:00 min

QUÉ DECIR

Finanzas muestra lo cobrado, la deuda vencida y los morosos. En Cobranza se ve cada pago; anular uno no lo borra: revierte el saldo y publica el evento. En Cuentas de acceso, el administrador restaura la cuenta de un alumno que olvidó la clave: vuelve a ser su documento y el sistema le obliga a cambiarla.

QUÉ MOSTRAR

Finanzas → Resumen / Morosos / Cobranza; Administración → Cuentas de acceso → Restaurar.

PASE A LA SIGUIENTE

Cierre: ver la frase final.

Frase de cierre (15 s): «Cinco procesos, tres roles móviles y una sola plataforma: ningún servicio lee la base de otro; todo pasa por transacciones locales y eventos, y si algo falla en medio, el sistema compensa.»

Hoja de respuestas · preguntas de defensa

Pregunta	Responde	Respuesta corta
¿Qué es el outbox y por qué?	Hermes	Guardar el dato y su evento en la misma transacción; un relé publica después. No se pierde ni se inventa un aviso.
¿Cómo evitan vender la misma vacante dos veces?	Andre	Una sola sentencia: UPDATE … WHERE ocupados < capacidad. Es atómica y devuelve cero filas si no hay cupo.
¿Qué pasa si el pago llega dos veces?	Andre / Witmer	Idempotencia: la clave del pago, la tabla eventos_procesados y el estado de la matrícula. No se cobra ni se procesa dos veces.
¿Por qué la matrícula espera una aprobación después del pago?	Andre	Porque pagar no es lo mismo que ser admitido: el administrador revisa antes de matricular. Mientras tanto el cupo queda reservado y no vence.
¿Por qué no usan una transacción global (2PC)?	Todos	Cada servicio tiene su base y queremos bajo acoplamiento. Usamos saga coreografiada con compensaciones.
¿Cómo se anula un cobro?	Witmer	Con otra transacción: revierte saldos, anula el comprobante y publica pago.anulado. No se borra nada.
¿Cómo saben quién es el usuario en cada servicio?	Hermes	Por el claim usuarioId del token; los endpoints internos lo traducen a docente, alumno o apoderado.
¿Qué pasa si Kafka se cae?	Todos	Los eventos esperan en outbox_eventos y el relé reintenta cuando vuelve.
¿Y si un servicio se cae?	Todos	Circuit breaker y respuestas controladas; la web muestra la franja de aviso y permite reintentar o volver a ingresar.
¿Cómo prueban todo esto?	Todos	Pruebas con PostgreSQL real, seis escenarios de punta a punta, scripts de demostración y auditoría de pantallas.
¿Qué falta?	Todos	Pasarela y correo reales, producción con pruebas de carga, prorrateo de pensiones y reembolso automático al rechazar una matrícula.

Si algo falla en vivo: «Quiero mostrarles justo cómo se comporta el sistema ante una falla» y pasen a la captura de respaldo. Si la web manda al inicio de sesión tras un reinicio, es el manejo de sesión: entren de nuevo y explíquenlo como una ventaja. Si aparece la franja «el sistema no responde», esperen un minuto y reintenten.